API CONTRACT · DEPLOYED AND SERVING

Course Builder

Three endpoints. **Plan** turns a topic into a course. **Build** queues every lesson as a video. **Status** reports progress. Everything is read-only from your side except `build`, which starts work but writes nothing into your system.

BASE URL https://content-queen-production.up.railway.app

TRY IT /demo/course-builder — the working planner. No credential needed: just type a topic. Rate limited to 5 plans an hour.

DISCOVER GET /api/v1 — public, machine-readable list of every route on this service.

Authentication

The same bearer token as the checkpoint API, required on every `/api/v1` route. Server to server — never from a browser, or you ship the token to every learner. (The `/demo` planner is deliberately open and rate limited, so you can try the idea before wiring anything.)

CREDENTIAL · TREAT AS A PASSWORD

HEADER `Authorization: Bearer <token>`

TOKEN `cq_o9UjsZ6_pknF10IUZwuZHM7Hl6MpJtDF`

Same token, same service as the in-video question API. **Do not put it in browser code** or commit it. If it leaks, tell Aroma; replacing it is one command and changes nothing else.

The three endpoints

ENDPOINT	DOES	COSTS
`POST /api/v1/courses/plan`	Topic in, full course plan out. Idempotent in the sense that it never changes anything — call it as often as you like.	**~60–90s.** One model call. No video spend.
`POST /api/v1/courses/build`	Queues every lesson in a plan as a video. Returns immediately with a `courseId`; the work happens over hours.	**~$1.50 and ~30 min PER LESSON.**
`GET /api/v1/courses/{courseId}`	Progress of a queued course.	Free. Poll every 30–60s.

THE MISTAKE THIS DESIGN EXISTS TO PREVENT

Planning is free and takes a minute. Building costs real money and takes hours. **Never chain plan straight into build.** A human must see the cost and approve it. Our API enforces this — `build` refuses unless you echo back the exact lesson count — but the meaningful guard is the screen you put in front of the user.

POST /api/v1/courses/plan

One required field. Everything else is optional — and if omitted, the planner picks defaults and lists them in `assumptions`.

REQUEST

```
curl -X POST https://content-queen-production.up.railway.app/api/v1/courses/plan \
  -H "Authorization: Bearer $CONTENT_API_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "topic":       "Using WhatsApp Business to take orders without losing any",  // required
    "duration":    "about 15 minutes",     // optional, free text
    "audience":    "shopkeepers with a smartphone",  // optional
    "description": "focus on catalogue and labels"   // optional
  }'
```

Response (200) — trimmed to one lesson; a real plan has 4–8

RESPONSE · APPLICATION/JSON

```
{
  "title":   "WhatsApp Business: Take Orders Without Losing Any",
  "summary": "One paragraph for your course catalogue.",
  "audience": "Small shopkeepers in Pakistan who already take orders on WhatsApp",
  "protagonist_scenario": "Ali runs a dairy shop in Gujranwala… lost eleven orders last month",
  "modules": [
    {
      "title":   "Seeing where the orders leak",
      "summary": "…",
      "lessons": [
        {
          "title":      "Spot the three places an order goes missing",
          "slo":        "Given a chat thread where an order was lost, the learner can name which leak caused it",
          "difficulty": "novice",        // novice | intermediate | advanced | expert
          "brief":      "The step of Ali's story this lesson covers — becomes the script brief",
          "question": {
            "stem":         "A customer messages at 9pm… Which leak is this?",
            "options":      ["Read but not recorded", "Incomplete details", "No update sent", "Cancelled"],
            "correctIndex": 0,
            "explanation":  "Why the common wrong choices are wrong, not just the right answer"
          }
        }
      ]
    }
  ],
  "assumptions": ["No audience was given, so…"],   // SHOW THESE TO THE USER
  "request":    { /* what you sent, echoed back */ },
  "estimate": {
    "lessons":              8,
    "estimatedCostUsd":     12,      // show this before any build button
    "estimatedBuildMinutes": 240,
    "estimatedWatchMinutes": 20
  },
  "plannedAt": "2026-09-11T12:04:11.902Z"
}
```

FIELD	USE IT FOR
`modules[].lessons[]`	**One lesson = one video.** This is the unit everything else counts in. Build your course structure from it directly.
`lesson.question`	The in-video question for that lesson. Same shape as the checkpoint API you already render — you can show it in the course outline before the video exists.
`assumptions`	**Show these.** They are how a user learns the planner guessed something, and the cheapest way to get a better plan is to let them correct one and re-plan.
`estimate`	Put on screen before any build control. It is the only thing standing between a curious click and $12.

POST /api/v1/courses/build

Send back the plan you got, the series it belongs to, and a confirmation of the lesson count.

REQUEST

```
{
  "plan":           { /* the whole object from /courses/plan, unmodified */ },
  "series":         "whatsapp-orders",   // required; lowercase a-z 0-9 . _ -
  "confirmLessons": 8                    // must equal plan's lesson count
}
```

Response (202 Accepted)

RESPONSE

```
{
  "courseId": "course-m1k2j3h4",   // store this; it is how you poll
  "series":   "whatsapp-orders",
  "queued":   8,
  "rejected": [],                  // lessons that could not queue, with reasons
  "items":    ["whatsapp-orders/spot-the-three-places-an-order-goes-missing", "…"],
  "status":   "https://…/api/v1/courses/course-m1k2j3h4"
}
```

Errors you must handle

STATUS	MEANING AND WHAT TO DO
`409 confirmation_required`	**Not an error — the guard working.** The body carries the real lesson count and the estimate. Show them, get a human yes, then resend with `confirmLessons` set to that number.
`400 bad_request`	Missing `plan`, or a `series` that is absent or not lowercase-slug shaped. The series is never guessed: a wrong one files an entire course in the wrong place and is only noticed after the money is spent.
`401 unauthorized`	Bad or missing bearer token.
`503 api_not_configured`	The service has no token configured and is failing closed. Ours to fix — retry later.
`500 plan_failed`	Planning failed upstream. Safe to retry once; nothing was spent.
Partial queue	A 202 with a non-empty `rejected` array means some lessons queued and some did not — usually a duplicate slug. Surface it; do not treat the course as fully started.

GET /api/v1/courses/{courseId}

RESPONSE

```
{
  "courseId": "course-m1k2j3h4",
  "lessons": 8, "done": 3, "failed": 0, "inProgress": 5,
  "items": [
    { "id": "whatsapp-orders/spot-the-three-places…", "topic": "…",
      "status": "done", "module": 1 }
  ]
}
```

Poll every 30–60 seconds. A finished lesson's video and its in-video question then come from the checkpoint API you already use: `GET /api/v1/videos/{videoId}/checkpoints`, where `videoId` is the part of the item id after the slash.

Limits that will affect your UI

LIMIT	DESIGN AROUND IT BY
Planning takes 60–90 seconds	A real progress state, not a spinner that looks hung. Our own prototype says "this takes about a minute".
Building takes hours	Treat a course build as a background job with a progress list. Never block a request on it.
We store no plans	**Persist the plan on your side as soon as you show it.** If the user navigates away, it is gone — we kept nothing.
Our build queue is not durable	If our service redeploys mid-build, the remaining lessons are lost and the course must be restarted. Show per-lesson status rather than a single percentage, so a stall is visible.
Videos render one at a time	Two courses started together take twice as long. Consider allowing one build at a time per user.
Lessons can fail QA and be remade	`failed` in the status response is real. Decide whether a course appears when partly ready.

How it fits what you have already built

YOU ALREADY HAVE	IT WORKS UNCHANGED
The in-video question popup	Courses built this way produce videos with the same checkpoints, from the same endpoint. Nothing to change.
The bearer token	Same token, same service, same header.
Your course/lesson tables	Map `modules[].lessons[]` onto them. We deliberately do not impose a schema — tell us yours and we will shape the plan to fit rather than the other way round.

> TWO THINGS THAT ARE GENUINELY UNFINISHED
>
> **Plans cannot be edited.** A user accepts a plan or re-plans with a better description. If editing matters to you, tell us — it changes the contract, because an edited plan has to come back to us.
>
> **Build durability.** Our queue does not survive a redeploy of our service. This is on our list; until it is fixed, a long course build is best run when nobody is shipping changes to the pipeline.